say it shorter

digitalisierung und programmierung · prof. dr. nicolas meseth

1. why anything shrinks
2. two tricks, and a zip
3. when redundancy runs out

part 1

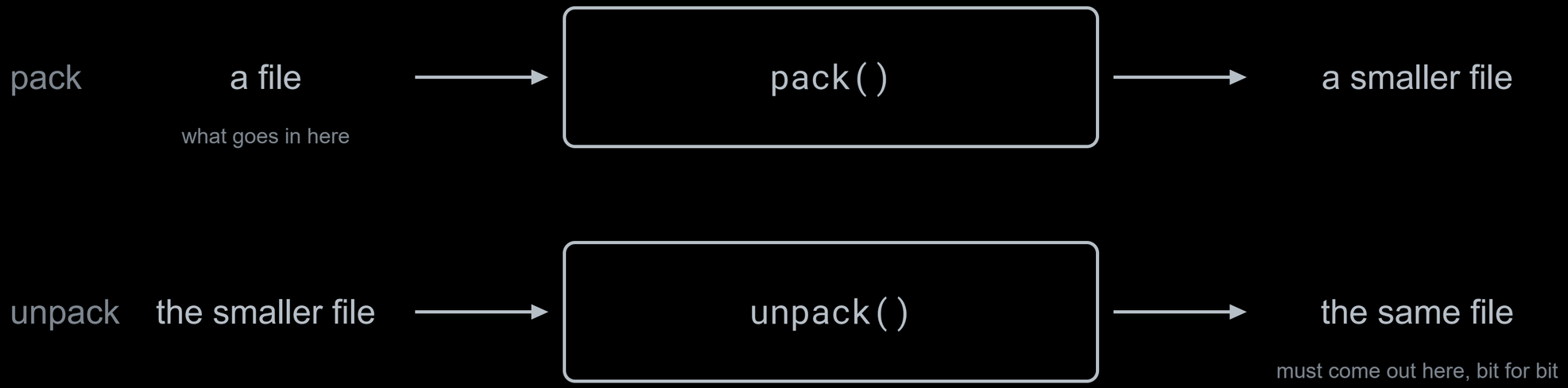
why anything shrinks

nothing is gone.
only the air between.



image: ai-generated (gpt-image-2)

compression as a box, twice



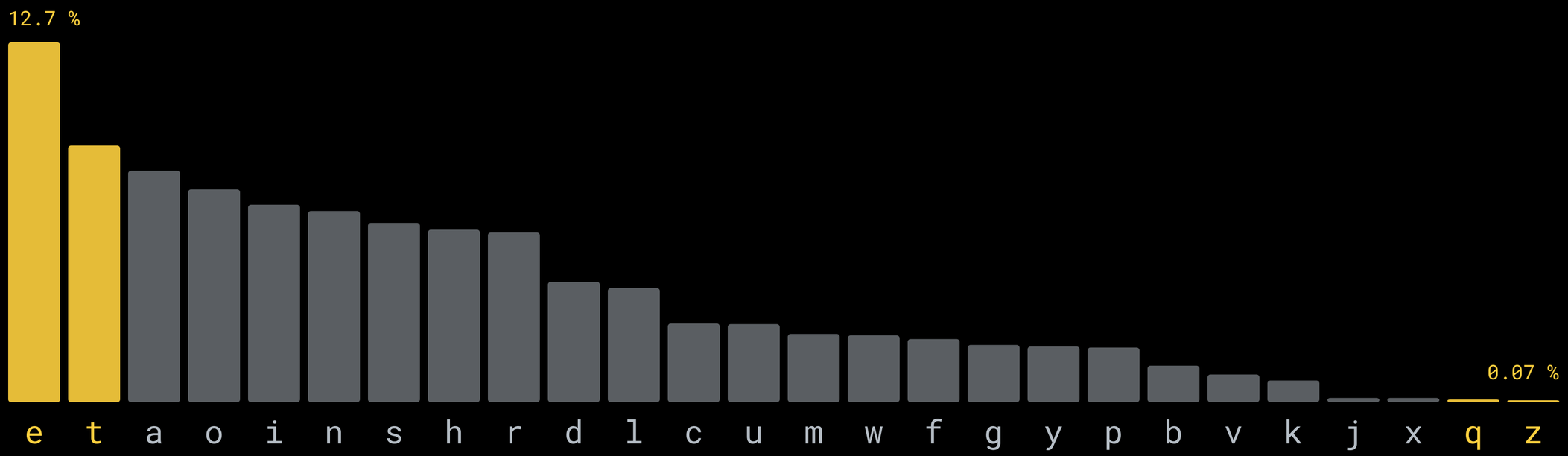
two boxes, input, processing, output. what goes into the first must come out of the second.

`the sun` shines and `the sun` warms and `the sun` sets

the same seven characters, three times, and a file written the usual way spells them out three times: contextual redundancy.

the usual way of writing things down wastes room. that room is the first kind of air.

a second kind of air: some letters come far more often



how often each letter appears in english text. the e comes round 180 times as often as the z: alphabetic redundancy.

a fixed code pays the same bits for the e and for the z. that difference is air too.

letter frequencies: lewand, cryptological mathematics, 2000

the word for both: redundancy

fewer bits for what surprises less

a stretch seen before: cheaper to name than to spell again (contextual)
a frequent letter: less surprise, so a shorter code (alphabetic)
random characters: full surprise every time, nothing to save

information, as in symbols and information: a measure of surprise

bits go to surprise. the rest is redundancy, and it can go.

part 2

two tricks, and a zip

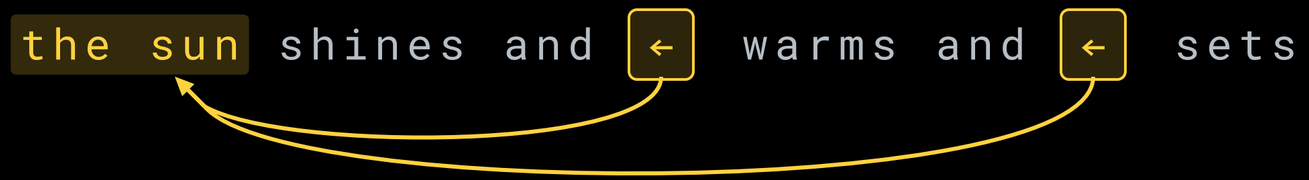
the sun shines and the sun warms and the sun sets

49 characters

say it once. then say where it was.

the sun shines and the sun warms and the sun sets

49 characters



say it once. then say where it was.

trick one: point back

the sun shines and the sun warms and the sun sets

49 characters



the sun shines and (19 back, 7 long) warms and (18 back, 7 long) sets

35 characters and 2 pointers. a pointer costs about as much as two characters.

say it once. then say where it was.

trick one: what it looks like in the file

raw: one byte per character

49 bytes

```
74 68 65 20 73 75 6e 20 73 68 69 6e 65 73 20 61    the·sun·shines·a
6e 64 20 74 68 65 20 73 75 6e 20 77 61 72 6d 73    nd·the·sun·warms
20 61 6e 64 20 74 68 65 20 73 75 6e 20 73 65 74    ·and·the·sun·set
73                                                 s
```

packed: pointers in yellow

41 bytes

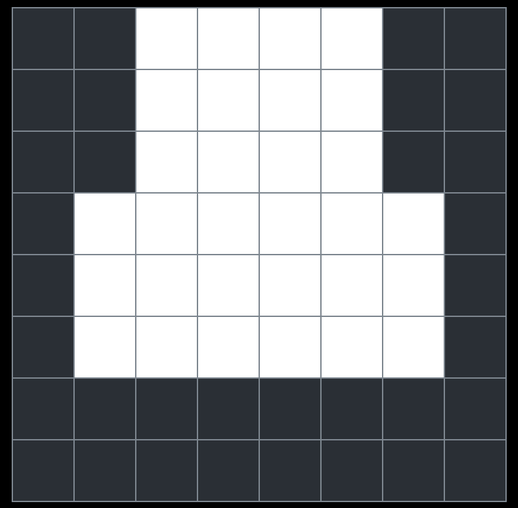
```
74 68 65 20 73 75 6e 20 73 68 69 6e 65 73 20 61    the·sun·shines·a
6e 64 20 ff 13 07 20 77 61 72 6d 73 20 61 6e 64    nd···· warms·and
20 ff 12 07 20 73 65 74 73                         ····· sets
```

ff 13 07 = marker, 19 back, 7 long · ff 12 07 = marker, 18 back, 7 long

a pointer is three bytes here: a marker, how far back, how long. 49 bytes become 41.

trick one on a picture: run length

8 x 8 points, one bit each: 8 bytes



row by row, count equal points:

```
2 dark, 4 light, 2 dark
1 dark, 6 light, 1 dark
8 dark
```

one byte per run: 1 bit colour, 7 bits length

6 bytes instead of 8

the bigger the areas, the more it saves.

trick one: run length, the bad case

the same 8 bytes



every point differs from its neighbour:

64 runs of length one

one byte each

a byte that described eight points now describes one

64 bytes instead of 8

every method makes some files bigger. yours too.

symbols and information: morse

e	·	the most frequent letter
t	–	
q	– – · –	one of the rarest
j	· – – –	

but it needs a pause after every letter: · · could be i, or e e

code systems: variable length

codes may differ in length

and still read without separators,

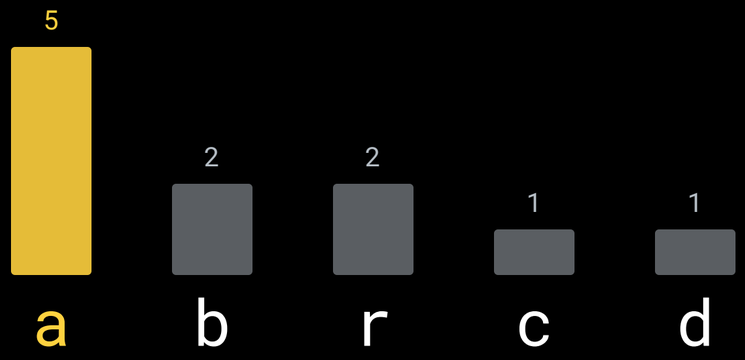
as long as no code starts another

what we left open: who gets the short codes, and why

both halves are known: codes of different length, the frequent one short. now the recipe.

a b r a c a d a b r a

11 letters, 5 different



fixed length, as in code systems:

```
5 letters -> 3 bits each
```

```
11 * 3 = 33 bit
```

the same code for every letter: the a pays as much as the d.

trick two: build the tree

the recipe

1 count every letter

2 join the two rarest into one node, count = sum

3 repeat until one node is left



c 1 + d 1 = 2

b 2 + r 2 = 4

2 + 4 = 6

a 5 + 6 = 11



always join the two rarest. that is the whole algorithm.

huffman, 1952

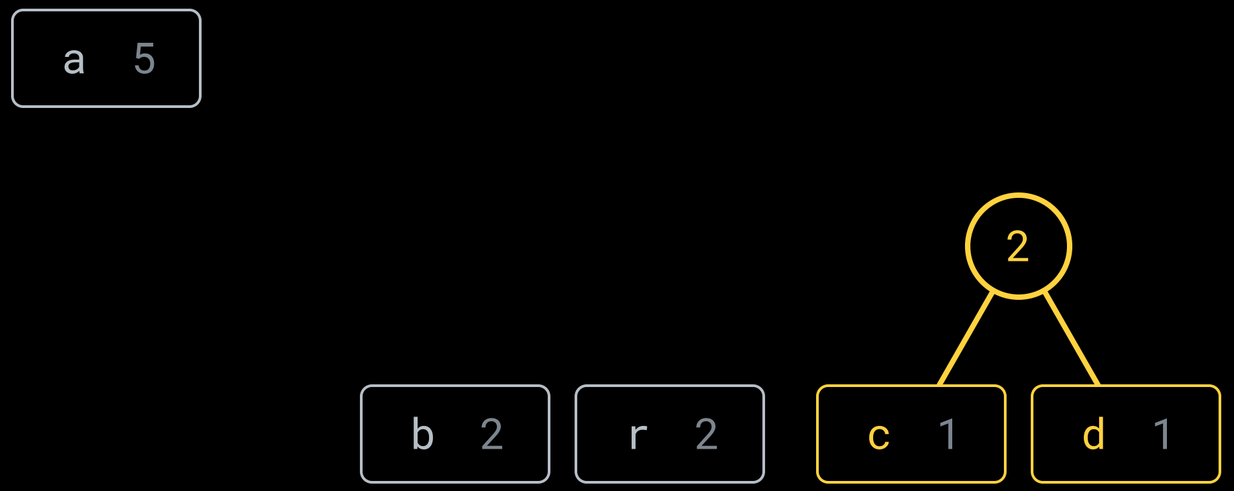
trick two: build the tree

the recipe

1 count every letter

2 join the two rarest into one node, count = sum

3 repeat until one node is left



c 1 + d 1 = 2

b 2 + r 2 = 4

2 + 4 = 6

a 5 + 6 = 11

always join the two rarest. that is the whole algorithm.

huffman, 1952

trick two: build the tree

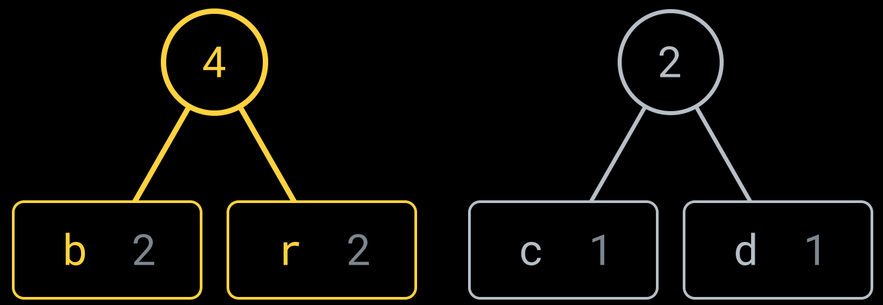
the recipe

1 count every letter

2 join the two rarest into one node, count = sum

3 repeat until one node is left





c 1 + d 1 = 2

b 2 + r 2 = 4

2 + 4 = 6

a 5 + 6 = 11

always join the two rarest. that is the whole algorithm.

huffman, 1952

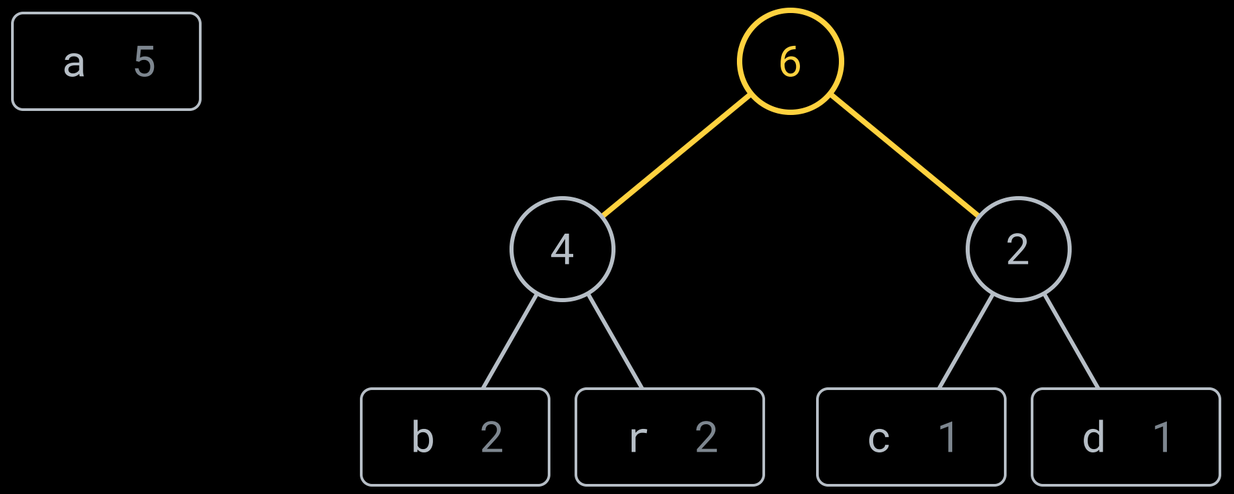
trick two: build the tree

the recipe

1 count every letter

2 join the two rarest into one node, count = sum

3 repeat until one node is left



c 1 + d 1 = 2

b 2 + r 2 = 4

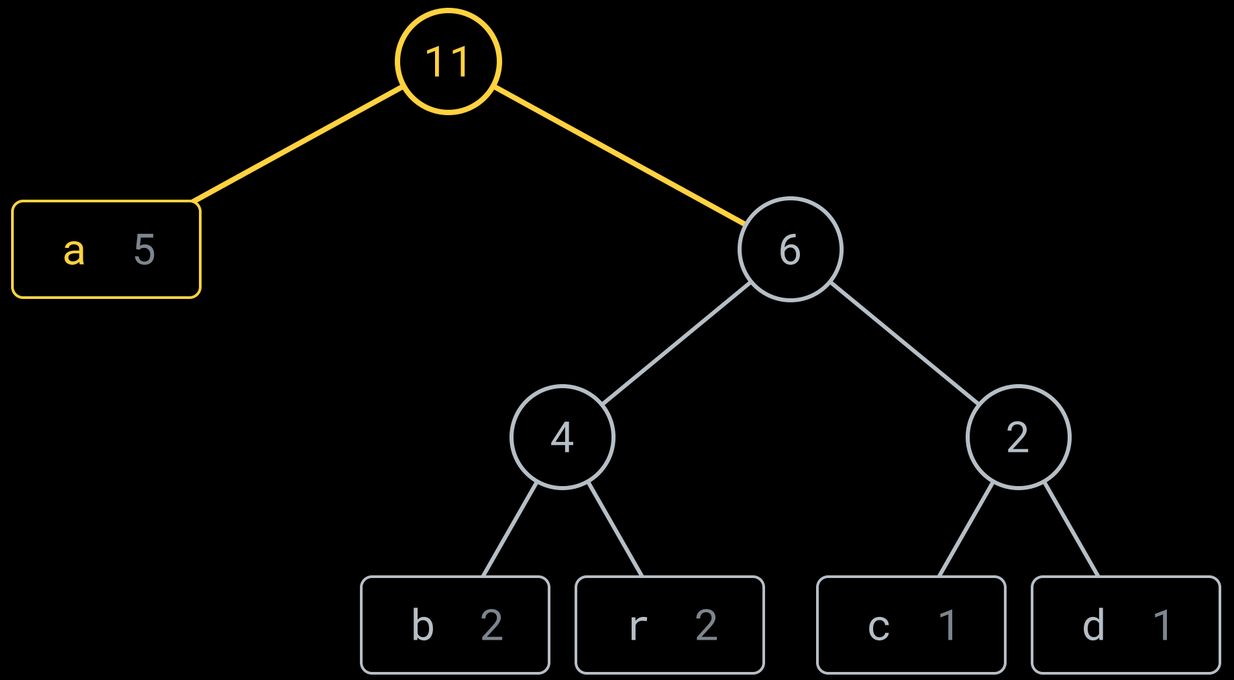
2 + 4 = 6

a 5 + 6 = 11

always join the two rarest. that is the whole algorithm.

huffman, 1952

trick two: build the tree



the recipe

1 count every letter

2 join the two rarest into one node, count = sum

3 repeat until one node is left

c 1 + d 1 = 2

b 2 + r 2 = 4

2 + 4 = 6

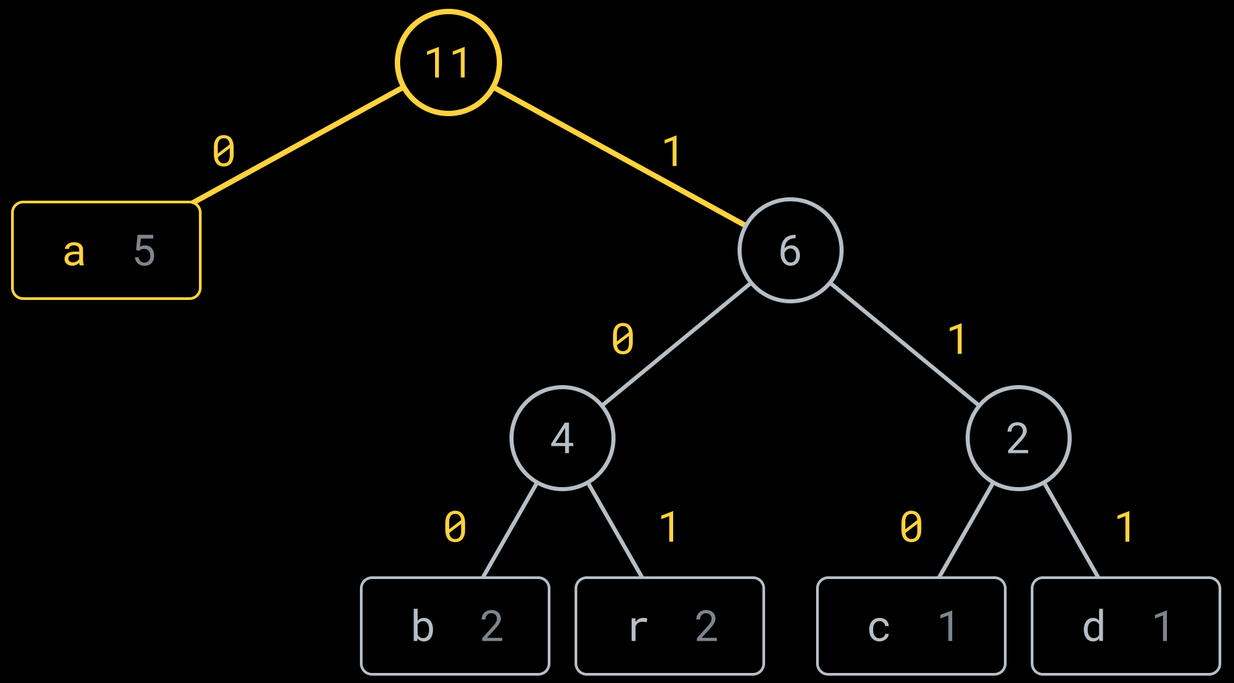
a 5 + 6 = 11

11: the root counts the whole word

always join the two rarest. that is the whole algorithm.

huffman, 1952

trick two: read the codes off the tree



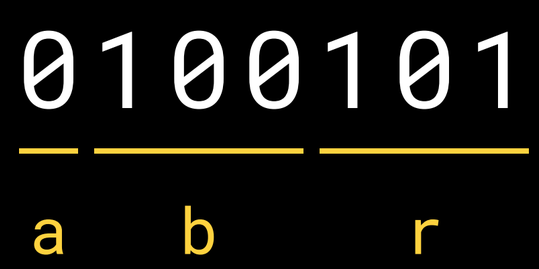
left 0, right 1:

```
a   0
b   100
r   101
c   110
d   111
```

```
0100 1010 1100 1110 1001 010
```

```
5*1 + 6*3 = 23 bit
```

23 bits instead of 33. nothing lost.



read from the left. as soon as a code is complete, the next one starts.

letters sit on leaves only

so no code is the beginning of another

0 is a leaf: nothing else starts with 0

no code is the start of another. the prefix trap from code systems, avoided by construction.

try it: squeeze your own text

the fastest transmission is the one where you have less to send. that sounds like a trick, and the first time you pack a text file and it comes back at half the size with every character intact, it feels like one. nothing was left out. what happened is that the usual way of writing things gives space away: some words come round again and again, some letters turn up constantly and others almost never, and after a q there is almost always a u. every one of those regularities can be said more briefly, and the text survives it untouched.

try: a sentence | one letter, 400 times | a paragraph | a log file | random letters | a bit pattern

fixed code 29 different characters, 5 bit each (as a file: 539 bytes)	2,695 bit
after trick one 269 characters stay, 69 pointers of 25 bit	3,070 bit 114 %
after both tricks plus a code table of 160 bits	2,977 bit 110 %

nothing repeats and nothing stands out: the pointers and the code table cost more than they save. the file grows.

---

trick one: point back (what the packer writes)

the fastest transmission is (28 back, 4 long)one where you have les(28 back, 3 long)o send. (37 back, 3 long)at sounds like a(72 back, 3 long)ic k, and(67 back, 5 long)fir(93 back, 4 long)im (68 back, 6 long)pack(37 back, 4 long)ext (27 back, 3 long)l(49 back, 3 long) (40 back, 3 long)it comes b(30 back, 5 long)t

trick two: count first (the codes for the letters that remain)

character	count	code	length
pointer	69	00	2 bit
␣	31	1110	4 bit
e	23	1011	4 bit
a	21	1001	4 bit
t	20	1000	4 bit
s	18	0110	4 bit
i	17	0100	4 bit

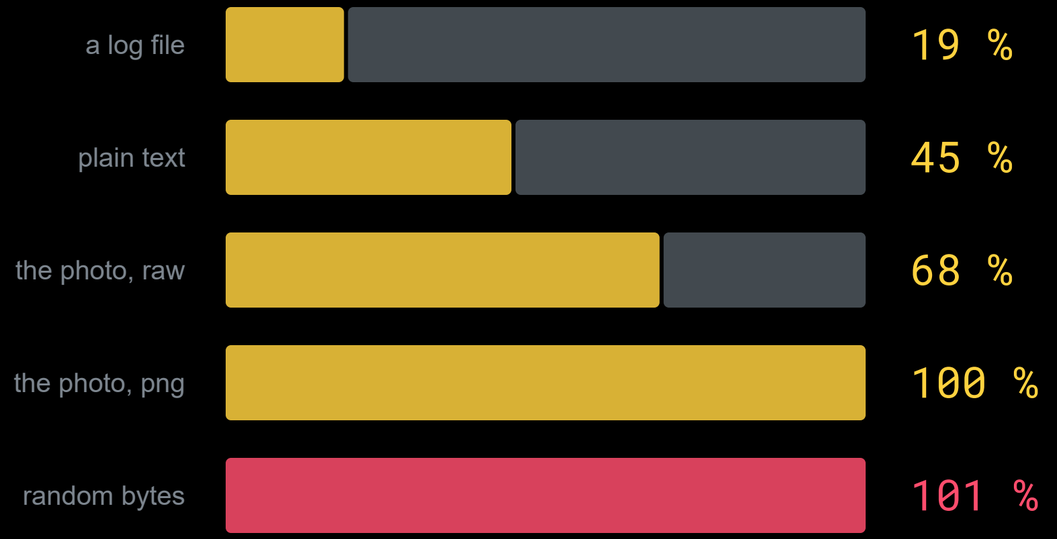
and that is a zip

zip

1 point back
lempel and ziv, 1977

2 count first
huffman, 1952

the pair is called deflate. png is the same pair with a picture in front.



every zip you ever opened did these two tricks. nothing else.

measured with python's zipfile, level 9, 13 sept 2026

why the random bytes could not shrink

every file of 3 bits

- 000
- 001
- 010
- 011
- 100
- 101
- 110
- 111

8

into

7

every file shorter than 3 bits

- (empty)
- 0
- 1
- 00
- 01
- 10
- 11

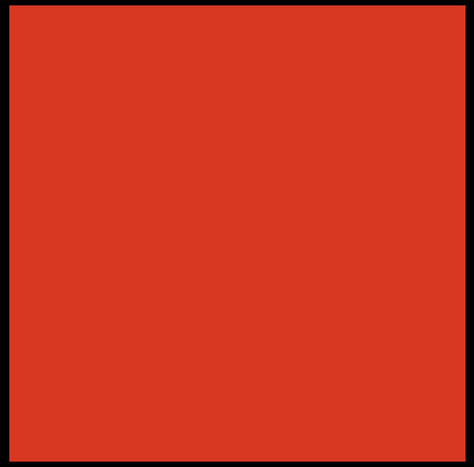
two of them have to share one shorter file, and then unpacking cannot be unambiguous

no method shrinks everything. counting says so.

part 3

when redundancy runs out

how much redundancy a picture has



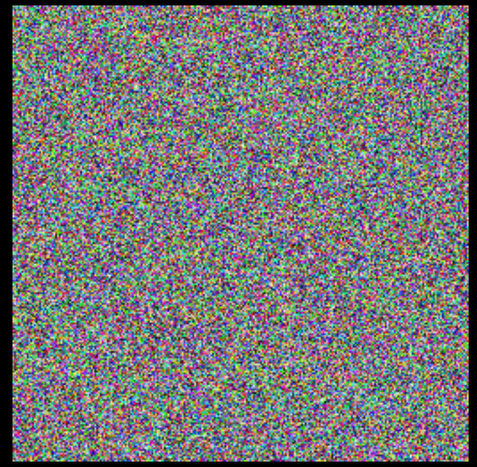
one colour

png · 569 bytes



the parrot

png · 93 107 bytes



random noise

png · 197 172 bytes

all three are 196 608 bytes raw. lossless packing takes the parrot to half; the noise it cannot touch.

real files, 256 × 256 points, sizes as measured · parrot: ai-generated (gpt-image-2)

what png does before it packs

twelve points from the parrot, one row

red values	251	250	249	249	251	251	251	250	249	249	248	249
minus left neighbour	251	-1	-1	0	+2	0	0	-1	-1	0	-1	+1

small numbers, mostly the same few: contextual redundancy between neighbouring points, air for both tricks

twelve points from the noise, the same row

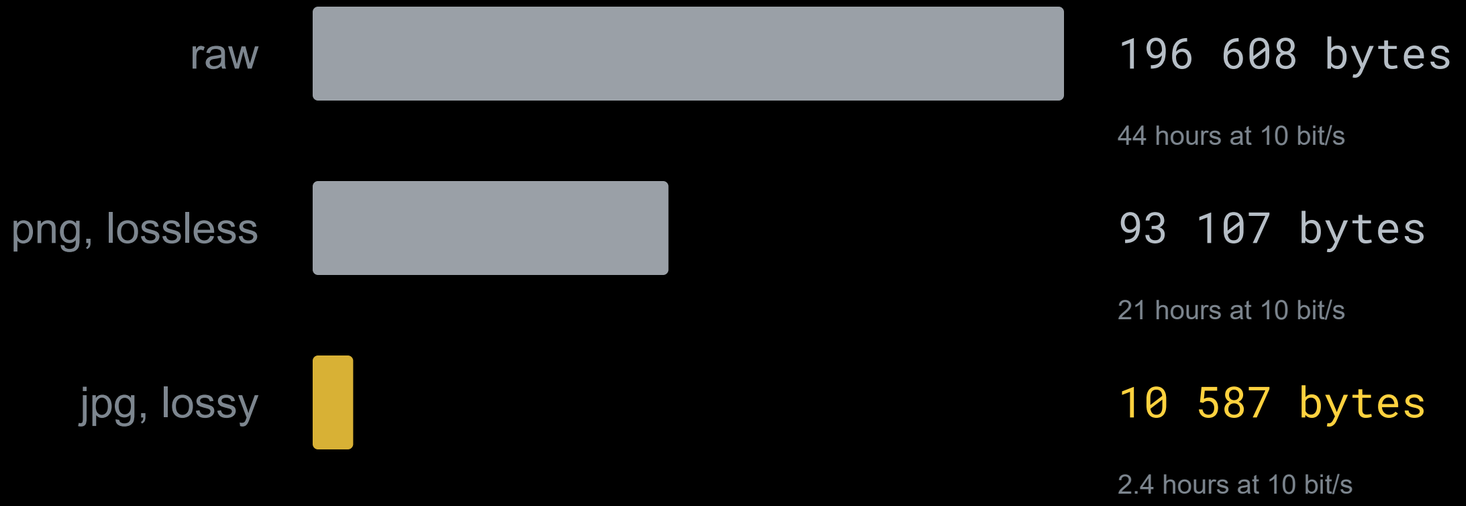
red values	208	213	184	203	181	229	178	181	102	159	225	101
minus left neighbour	208	+5	-29	+19	-22	+48	-51	+3	-79	+57	+66	-124

the differences are as random as the values: nothing gained

predict each point from its neighbour, keep only the surprise. then deflate.



the same 256 × 256 points, three ways of writing them down



lossless gets you to half. lossy gets you to a twentieth, and it is not the same picture.

the same parrot, four times



png · 93 107 bytes

lossless



jpg · 10 587 bytes

quality 75



jpg · 3 081 bytes

quality 10



jpg · 2 032 bytes

128 × 128 points, quality 25

still a parrot. and now it fits.

real files, sizes as measured · image: ai-generated (gpt-image-2)



the eye misses

fine differences in colour
it sees edges in brightness sharply, edges in colour blurred

small detail in a busy area
feathers, grass, gravel: the texture is noticed, not the single point

tiny steps in brightness
250 and 251 look the same

jpg keeps what you would see and rounds the rest away



the ear misses

a quiet tone while a loud one plays
at the same time and close in pitch: the loud one masks it

very high frequencies
above what the ear can hear at all

a quiet sound just after a loud one
in any pitch: the ear needs a moment to recover

mp3 keeps what you would hear and rounds the rest away

lossy means: a model of the senses decides what may go. not a worse packer, a different bargain.

images: ai-generated (gpt-image-2)

a one-way street

lossless: unpack, and you hold the original
lossy: unpack, and you hold something that looks like it

from the 2 kb parrot, the 93 kb never come back.

lossless or lossy

must exactly this file arrive? then lossy is out.
must a presentable picture arrive? then it is your strongest lever.

the task decides, not the tool. and ask before you spend an hour optimising.

the receiver has to know

which method
which settings
written into the specification, next to preamble and character code

packed data does not explain itself.

1 zip three files of your own and explain the three ratios
2 bring the parrot under 2 KiB, and write down what you gave up
3 put method and settings into your specification

two kilobytes.
still a parrot.



photo: parrot, 128 × 128 points, 2 032 bytes, shown point for point · image: ai-generated (gpt-image-2)